

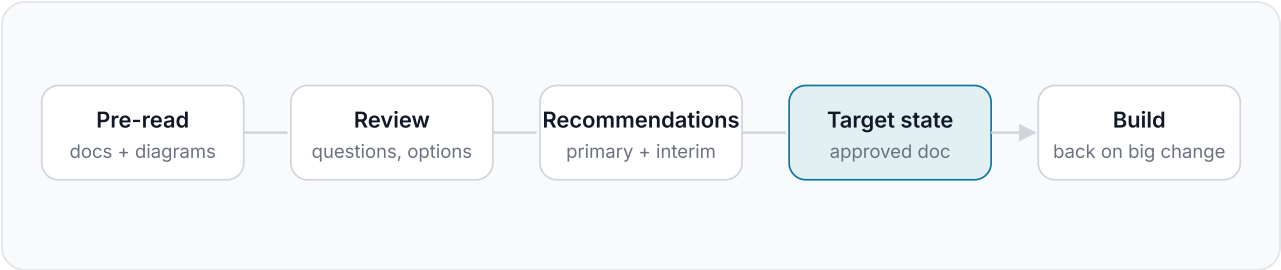
How to Prepare for an Architecture Review

What to bring to a cloud architecture review so it's one meeting, not three: entry criteria, the diagrams reviewers need, the cross-cutting concerns people forget, and how to handle open questions.

September 26, 2026 · 4 min read
<https://brianpfeil.com/architecture/prepare-for-an-architecture-review/>

Most architecture reviews that go badly go badly for the same reason: the team arrives with a good design and an incomplete story. The reviewers spend the hour discovering gaps instead of discussing trade-offs, and everyone books a second meeting.

This is what I'd tell any team preparing for one. None of it is specific to a company; it's what reviewers everywhere are trying to find out.



1. Meet the entry criteria

Before anyone looks at a diagram, reviewers want five facts. If any are missing, that's the first thing they'll ask about.

Bring	Why they ask
The business need and a short executive summary	Every trade-off is judged against what the system is <i>for</i> .
The proposed architecture	Obviously — but as diagrams, not prose (see below).
Whether it's regulated	Changes the controls, the evidence and sometimes the region.
The data classification	Drives encryption, access, retention and logging.

Bring	Why they ask
The business criticality	Drives availability targets and whether DR is needed at all.

2. Bring diagrams reviewers can read

A good diagram answers questions before they're asked. Draw it in your cloud provider's reference-architecture style, number the steps of the main flow, and show the boundaries: internet, your company, on-premises, other clouds, partners and SaaS; accounts, regions, networks, availability zones and subnets; devices and users.

One diagram can't show everything. I ask for four:

1. **The core solution** — the request and data flow, numbered.
2. **Build and deploy** — the pipeline, from commit to production.
3. **Availability, backup and recovery** — what fails over to where.
4. **The vendor or partner lifecycle**, if there is one — who ships updates and how they arrive.

There's more on this in [Diagrams and decision records reviewers trust](#).

3. Cover the cross-cutting concerns

These are the topics teams leave out, and each one is a reason to be sent back. Put an answer to every row on a page or a diagram.

Concern	The question behind it
Infrastructure as code	Can the whole thing be rebuilt from a repository?
CI/CD	What happens on every commit?
IAM, per component	What can each piece do, and nothing more?
Secrets	Where are they, and do they rotate?
App configuration	Where does it live, and how does it change safely?
Logs, metrics, alarms	Who finds out when it breaks?

Concern	The question behind it
Backups	What's backed up, how often, and has a restore been tested?
Self-healing	What recovers without a human?
Disaster recovery	Recovery time and point objectives, and how you'd meet them.
Versioning	APIs, schemas, artefacts.
Audit	What evidence exists, and where?
Cost	Including idle cost and data transfer.

4. Mark every service's status

If your organisation keeps a list of approved cloud services, mark each service on the diagram as **approved**, **approved with conditions**, **not approved** or **not yet requested**, and bring a list of the gaps. It turns a vague worry ("are we allowed to use that?") into a tracked action.

5. Justify anything that isn't managed

Reviewers will push anything that isn't fully managed and pay-per-use up the ladder: servers toward containers, containers toward serverless, custom code toward a managed service. That's fine — sometimes there's a real reason. But come ready to show you understand your workload in detail: its traffic shape, its state, its runtime, its limits. "We've always run it on a VM" is not a reason.

6. Leave with decisions, not just notes

- Keep a **shared parking lot** for questions nobody can answer in the room, and a chat channel for follow-ups, so the meeting doesn't stall.
- Expect **recommendations in two tiers**: the primary target, and an interim step you can ship now on the way there.
- Once you've picked an option, write the **target-state architecture document** and get it approved *before* you build. Come back only when something significant changes.
- Record the big choices as [architecture decision records](#).

- Store every artefact, versioned, in one predictable place linked to the ticket that tracks the work.

The more completely your diagrams and documents capture the agreed architecture, the more likely it is that the thing that gets built matches it. That's the whole point of the exercise.

Before you send the invite

- ☐ Executive summary, business need, regulation, data classification, criticality
- ☐ Core, CI/CD, availability/DR and (if relevant) vendor diagrams, numbered and bounded
- ☐ An answer for every cross-cutting concern in the table above
- ☐ Every service marked with its approval status; gaps listed
- ☐ A reason for everything that isn't managed and pay-per-use
- ☐ A cost estimate with idle cost and data transfer
- ☐ The [review checklist](#) answered